

Inigo Quilez :: articles :: volumetric sort - 2006



Intro

Let's say you have a large set of small objects that you want to alpha blend in a front to back or back to front order. Let's say the position of these objects is constant. And let's also assume that your objects are all positioned along a 2d or 3d grid, or close to a grid. This can be the case when rendering a point cloud of gaussian splats for example, as we usually do for laser scanned data. Then, you can very easily sort this objects at virtually zero performance cost, and this article will explain you how.

It might look like the premises are too restrictive, not applicable to "real life". However, imagine you have a field of grass, and you want to draw the blades in back-to-front order. You can probably afford aligning them into a 2D grid, and might be apply random scale and orientation to break the regularity. Or may be you have a cloud rendering engine using billboards, and you have to sort them also to properly alpha blend the particles. Or you could even have a point-cloud viewer showing some nice Julia sets coming from a 1024^3 voxel (like the one in the end of this article).

In 2D

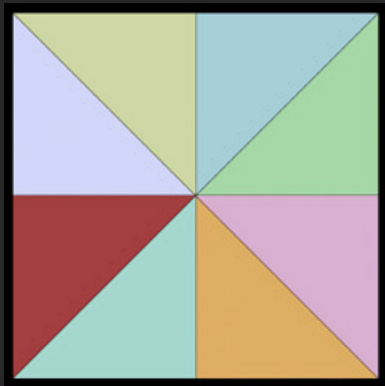
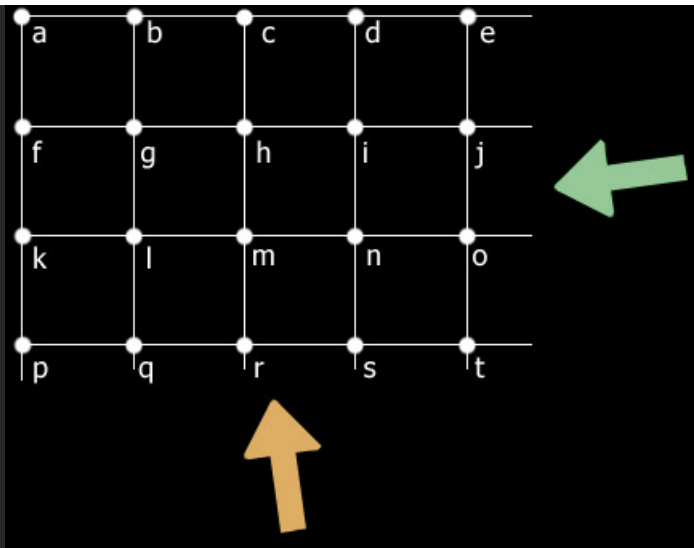
To explain the technique, let's first think about the problem in 2D. Let's say you have a grid of objects like the one below. Now let's say you are looking at this grid of objects from the view point indicated by the orange arrow in the diagram. Now, try to agree with the fact that given this situation you could draw your objects line by line, starting from the line **a**, **b**... until the line **p**, **q**...

That's obviously because we are looking roughly from bottom to top. Note also that because we are a bit skewed to the left, we better draw the objects from "left to right" within each line; ie, **a**, **c**, **d**, ... instead of ..., **e**, **d**, **c**, **b**, **a**.

Good. In a very similar way, the correct order to render the objects in the grid for a viewpoint like the one indicated with the green arrow should be left to right for the columns, and then bottom to top within each column: ..., **p**, **k**, **f**, **a**, ..., **q**, **l**, **g**, **b**, ...

So it's quite simple. We can determine the order just based on the view vector. If instead of "left to right" and "bottom to up" we use $+x$, $-x$, $+y$ and $-y$, we can easily see that there are 8 different possible orders: $\{ +x+y, +x-y, -x+y, +x-y, +y+x, +y-x, -y+x, -y-x \}$ (basically we have 4 options for the first axis ($+x$, $-x$, $+y$, $-y$) and there is only 2 remaining for the second ($+x$, $-x$ or $+y$, $-y$, depending on the first option)).

The transition between one order and another is done on half quadrants, as you can see. The figure showing 2D square split in 8 sections shows the areas where the same order is valid (you can see the orange and green areas corresponding to the arrows we used as example in the previous diagram). The trick now is clear: precompute 8 index arrays and save them in video memory, one for each possible order. For each rendering pass, take the view direction and compute which of the orders is the good one, and use it to render. So, we basically skip any sorting time, and also bus traffic between the CPU and the GPU. The only drawback is that we need 8 copies of the index arrays in memory instead of 8, and as we will see immediately, it is even worse in 3D... But again, is so cool to have zero sorting cost!



When the view vector stays in any point of a colored area, the order is the same.

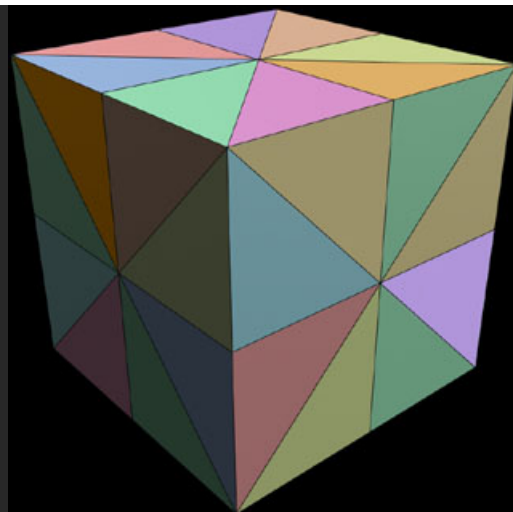
In 3D

In 3D the situation is quite the same, we only have one axis more. The difference is that now the amount of possible orders is quite bigger. For the first axis's order we have 6 options (-x,+x,-y,+y,-z,+z), for the second we have 4 (assume we chose -x, then we still have -y,+y,-z,+z) and 2 for the last axis (assuming we chose +z, we still have -y and +y). So that's a total of 48 possibilities! This can be a lot of video memory depending on the application. There are some simple tricks to help of course. For example, we keep the 48 copies in system memory and just upload the one we need.

Assuming frame to frame coherence, this should happen not too often. We can even have a small thread running in parallel to the rendering just calculating the index array instead of precalculating and storing it in system memory. We can even anticipate the camera movement and precompute (asynchronously) the next expected index array.

Another trick is to have a top level grid to sort cells of objects, and then let random ordered drawing of the objects in the cell. If the objects were a field of grass, this can work pretty well. Or even, if we already have an octree data structure to do frustum culling and occlusion queries on the dataset, we can sort the octree nodes with this technique and then do standard CPU sorting in the visible node, or even have precomputed index arrays per-node.

Now the view vector can belong to 48 possible sections in the surface of a cube, as shown in the picture below.



In 3D, we have 48 areas for the view vector.

Implementation

To finish the article, a bit of code to show how you can get the order index (from 0 to 47) from the 3D view vector. There is probably a more simple (read compact) way to do it.

```
int calcOrder( const vec3 & dir )
{
```

```
int signs;

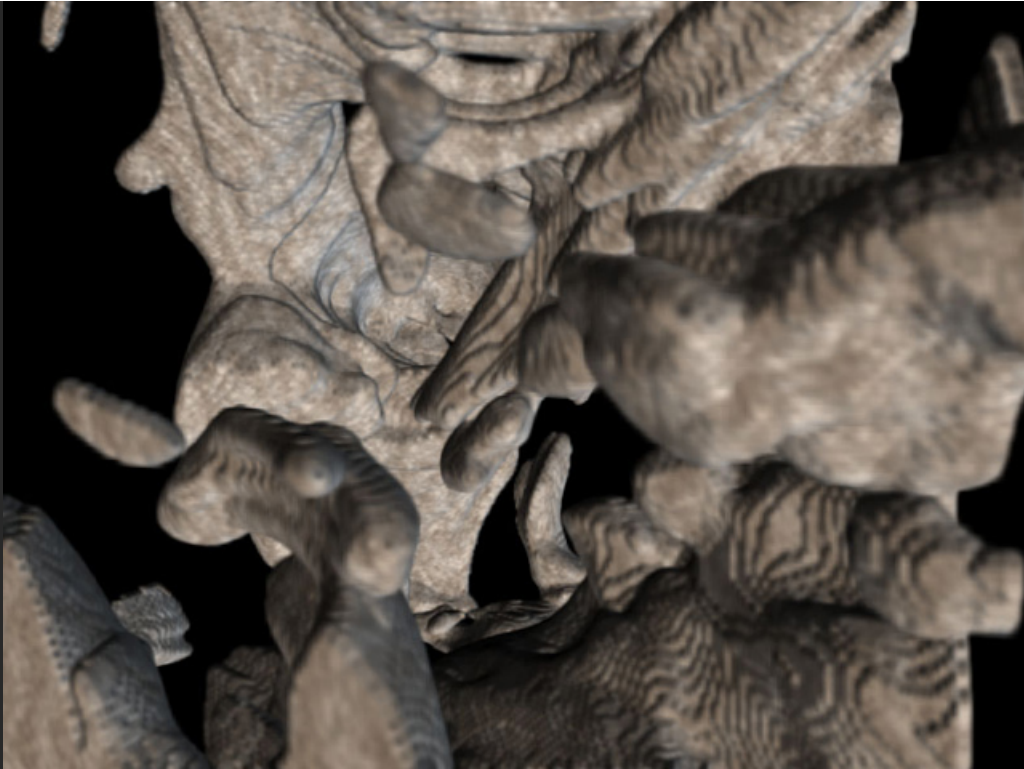
const int  sx = dir.x<0.0f;
const int  sy = dir.y<0.0f;
const int  sz = dir.z<0.0f;
const float ax = fabsf( dir.x );
const float ay = fabsf( dir.y );
const float az = fabsf( dir.z );

if( ax>ay && ax>az )
{
    if( ay>az ) signs = 0 + ((sx<<2)|(sy<<1)|sz);
    else      signs = 8 + ((sx<<2)|(sz<<1)|sy);
}
else if( ay>az )
{
    if( ax>az ) signs = 16 + ((sy<<2)|(sx<<1)|sz);
    else      signs = 24 + ((sy<<2)|(sz<<1)|sx);
}
else
{
    if( ax>ay ) signs = 32 + ((sz<<2)|(sx<<1)|sy);
    else      signs = 40 + ((sz<<2)|(sy<<1)|sx);
}

return signs;
}
```

Examples

These are a few images rendered in realtime with gaussian splatting, using the technique presented in this article for sorting the splats for correct blending. I ran these experiments in 2005, and all splat positions were precalculated and, and stored in large static vertex buffers. There were 48 static index buffers precomputed as well and the right one was chosen at run time based on view angle, as explained in this article.



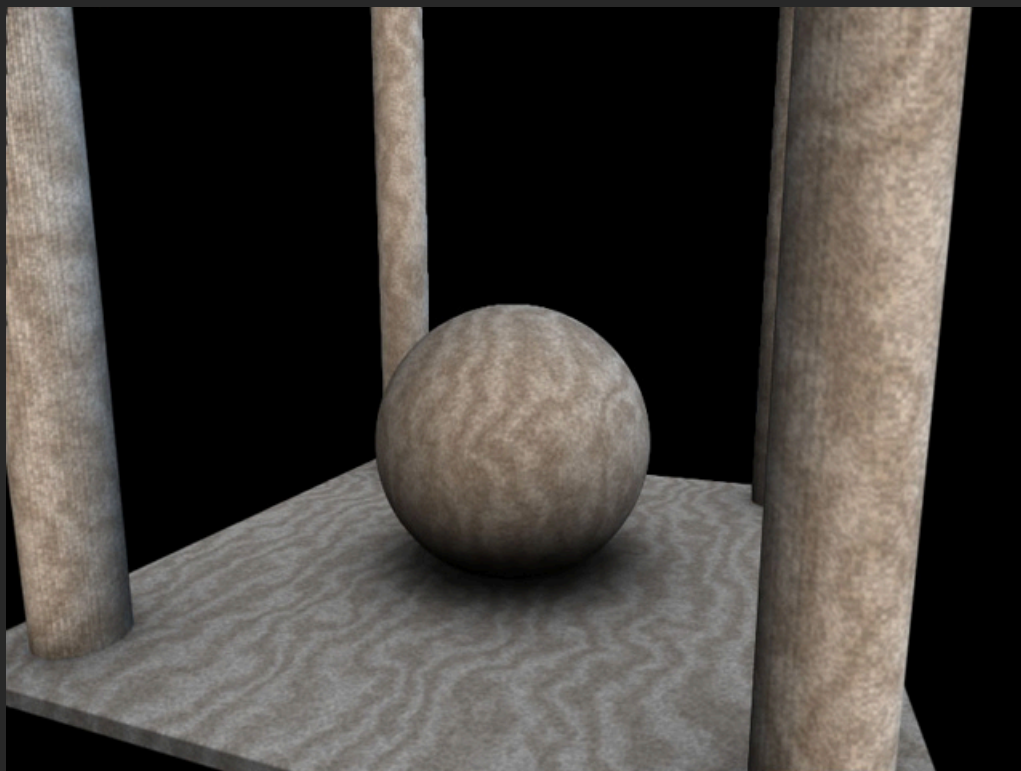
3D Noise volume



Julia set



3D Noise



SDF primitives